React – Exam Project: Functional Guide

Deployed	https://react01-eight.vercel.app/
GitHub repository:	https://github.com/Martin7n/react01

Table of Contents

1. Project Overview....1
2. User Access & Permissions....2
Guest (Not Authenticated)....2
Authenticated User....2
3. Authentication & Session Handling....2
Authentication Flow....2
Session Persistence....3
4. Routing Structure....3
Route Guards Logic....3
Main Routes....3
Nested & Parameterized Routes....3
5. List → Details Flow....4
Catalog / List Page....4
Details Page....4
6. Data Source & Backend....4
7. Data Operations (CRUD)....4
8. Forms & Validation....5
Forms Used....5
Validation Rules....5
9. React-Specific Techniques....5
Hooks & Component Lifecycle....5
Context API....5
Component Styling – tailwind and external css....5
10. Typical User Flow....6
11. Error & Edge Case Handling....6

1. Project Overview

- **Application Name:** SevenTech Node Platform
- **Application Category / Topic:** Technical Documentation & Track Catalog Platform
- **Main Purpose (2–4 sentences):**
  SevenTech acts as a high-performance, secure portal for technical teams to organize, track, and collaborate on multi-author documentation nodes. It solves the performance bottleneck of heavy

database joins by using a client-side array management mapping strategy to calculate content tracking layers instantly. Furthermore, it enforces airtight author validation rules via server-side database configurations to preserve content integrity on live environments.

---

2. User Access & Permissions

Guest (Not Authenticated)

Describe what an unauthenticated user can access:

- **Available pages / routes:** /, /catalog, /catalog/:id, /article/:id, /forgot-password, /login, /register, and the catch-all * (NotFound error layout).
- **Public data shown:** Full browsing rights to the category catalog grid cards, structural article density numbers, metadata publication stamps, and full-text reading permissions to documentation articles.

Authenticated User

Describe what a logged-in user can access:

- **Main sections / pages:** Private member area at /dashboard featuring custom analytics telemetry cards and a personalized chronological stream feed.
- **Details pages:** Interactive category dashboards (/catalog/:id) and full document views (/article/:id) showing specific author profile details.
- **Create / Edit / Delete actions:**
- *Creation:* Allowed to create new category elements and publish new articles under any track node.
- *Modification & Deletion:* Strictly restricted to assets they created themselves. Authorized profiles can update or delete their own articles or category nodes, verified directly via active authentication signatures.

---

3. Authentication & Session Handling

Authentication Flow

- **What happens when the app loads:** The application boots up and initializes the global <AuthProvider>. It inspects localStorage for pre-existing session criteria (access_token and refresh_token).
- **How authentication status is checked:** A background verification routine reaches out to Supabase using the cached access_token. If an expiration code is caught, it attempts an automated update sequence using the refresh_token.

- **What happens on successful login or registration:** The server issues an updated authentication JWT token pair. The token details are saved to `localStorage`, the local memory state `user` variable updates, and the client automatically routes the user to the catalog canvas.
- **What happens on logout:** The application executes a unified `session_clear()` cleanup macro. It clears the storage parameters, overrides the `user` context state to `null`, and navigates the browser cleanly to the login screen.

Session Persistence

- **How is the user session stored?:** Kept inside local React memory Context for immediate component access and synced permanently into `localStorage` across browser tab instances.
- **How is automatic login handled after a page refresh?:** During the `useEffect` initialization mount cycle of the context provider, the app pulls the access token out of storage, checks its integrity, and populates the reactive context state memory block instantly before mounting visible views.

---

4. Routing Structure

Route Guards Logic

- **How is routing split between guest-only and private routes?:** Implemented via a specialized `<ProtectedRoute>` layout wrapper. If the user object evaluates to `null` while attempting to access a guarded path, the route gatekeeper short-circuits the render and uses `<Navigate to="/login" replace />` to redirect them away.

Main Routes

- **Number and type of top-level pages:**
- `/catalog` (Public directory index grid layout)
- `/catalog/:id` (Public specific details channel view)
- `/article/:id` (Public full-text technical documentation node)
- `/dashboard` (Guarded private user control station stream)
- `/login` & `/register` (Public identity forms entries)
- `/forgot-password` (Static humorous local memory help playground canvas)

Nested & Parameterized Routes

- **Is there nested routing?:** No, layout headers are mounted consistently at the global route engine node layer.
- **Which routes use URL parameters?:** `/catalog/:id` (capturing category track UUID parameters) and `/article/:id` (capturing specific article row entry UUID references).

---

5. List → Details Flow

Catalog / List Page

- **What data is displayed?:** Category title strings, description overview fields, thumbnail images, real-time article count badges, and conditional subscription indicator status tags (✓ Followed).
- **How does the user interact with the list?:** Built using a **Controlled Form input search bar** that binds its current field parameters natively to a searchQuery state variable, instantly filtering matching grid cards on every single keystroke.

Details Page

- **How is navigation triggered?:** Triggered by clicking a catalog card wrapper running a declarative navigate(/catalog/${category.id}) hook.
- **What data is received via route parameters?:** The unique database string UUID parameter of the category node, which is parsed by useParams() and passed down to fetch target articles.

---

6. Data Source & Backend

- **Backend Type:** A real production backend database cluster powered by **Supabase**, connected natively via a centralized, authorized fetch engine (**PostgREST REST API**) running behind an explicit header key token manager.

---

7. Data Operations (CRUD)

The CRUD capabilities are securely modeled around the **articles** collection:

- **Create (POST):** Authorized profiles fill out a form modal. The system fires a POST request to the /rest/v1/articles collection endpoint. Upon a successful write, it captures the newly generated unique ID and attaches it to the parent category track array.
- **Read (GET):** Handled inside ArticleRead.jsx. It issues a clean GET request matching the URL route parameters to pull the specific documentation text block, followed by a parallel query layer execution to resolve the author's public profile email.
- **Update (PUT/PATCH):** Authors click an "Edit" modifier trigger button. The app routes a PATCH request targeting articles?id=eq.${id} with a JSON update body payload. The local state arrays are synchronized directly in client memory upon a successful network callback.
- **Delete:** Executed via a database DELETE request path targeting articles?id=eq.${id}. The application then strips the corresponding structural identifier references out of the sibling category array parameters using a client-side .filter() method before patching the layout update back to the database.

---

8. Forms & Validation

Forms Used

1. **User Sign In Form** (Uncontrolled layout utilizing FormData)
2. **Account Registration Form** (Uncontrolled layout utilizing FormData)
3. **Category Creation Modal Form** (Uncontrolled layout utilizing FormData)
4. **Article Publishing Workspace Form** (Uncontrolled layout utilizing FormData)
5. **Live Search Filter Bar** (Fully **Controlled Form** running on a dynamic text input)

Validation Rules

- **Field name and rules:** name (Category modal input: Must be a string with a minimum length of 3 trimmed characters).
- **Field name and rules:** description (Category/Article forms input: Must be a string with a minimum length of 3 trimmed characters).
- **Field name with multiple validation rules:** imgUrl (Optional image string fields: Can be an empty slot, but if a value string is supplied, it must validate as a secure URL format containing a minimum of 5 characters).

---

9. React-Specific Techniques

Hooks & Component Lifecycle

- **Which hooks are used, and where?:** useState (for managing modals, tracking toggles, and searching), useEffect (for triggering layout data fetching and synchronization queries), useParams (for reading URL keys), useNavigate (for routing redirects), and useContext (for binding user auth states).
- **Give one example of mount / update / unmount behavior in your app:** Inside CategoryPage.jsx: On **mount**, the useEffect loop executes and pulls category details from Supabase. On **update** (if the user logs in mid-session or switches category routes), the effect re-evaluates the active subscription state overlays. On **unmount**, a cleanup variable (isMounted = false) flips inside memory to cancel asynchronous network triggers and prevent memory leaks.

Context API

- **What state/data is shared through Context?:** The user authentication state profile, the application loading flags, and the main session handler parameters (loginUser, logoutUser).
- **Which components consume it?:** Navigation.jsx, Catalog.jsx, CategoryPage.jsx, Dashboard.jsx, and ProtectedRoute.jsx.

Component Styling – tailwind and external css.

- **How are components styled?:** The base application layout uses utility-first design primitives from the **Tailwind CSS** framework, while the responsive fixed global menu controls are decoupled

cleanly into an **external CSS stylesheet file (`Navigation.css`)** to manage cross-platform transformations smoothly.

---

10. Typical User Flow

1. **Entry & Exploration:** A guest enters the `/catalog` dashboard view, interacts with the **controlled search form** to live-filter specific topic tracks, and reads articles without creating an account.
2. **Identity Onboarding:** The visitor clicks the navbar links, fills out the registration form, and registers an account. An automated database trigger copies their public information seamlessly into the `profiles` data layer.
3. **Content Creation:** The logged-in user hits a topic track dashboard, clicks the secured button options to create a custom category node or post an article, and binds their metadata automatically as the author.
4. **Dashboard Tracking:** The member moves to their private `/dashboard` center page where their **Personalized Live Feed** aggregates a custom timeline of every article change published across the specific tracks they follow.

---

11. Error & Edge Case Handling

- **Authentication errors:** Caught directly by a centralized network interceptor inside `apiRequests.js`. If a request encounters a `401` or `403` token expiration, the handler wipes the local storage and routes the tab straight to a custom `/unauthorized` checkpoint screen.
- **Network or data errors:** If a user types random symbols into an article link route, the app intercepts the resulting `400 Bad Request` database compilation error before it can cause a page crash, safely routing the reader over to a terminal-themed `/not-found` 404 page.
- **Empty or missing data state:** Grid mapping components use short-circuiting logic expressions. If a category channel contains no database rows, the canvas suppresses errors and renders a smooth gray inline text placeholder reading `"Empty Node"` or `"No recent data streams available."`
